AURORADB / DATABASE SYSTEM

软件工程文档

需求分析 · 概要设计 · 详细设计（UML）· 测试设计与测试用例 · 分工

数据库原理课程大作业方案一　AuroraDB v1.2.0

11
用例

55
测试用例

279
自动化测试通过

项目	内容
文档名称	AuroraDB 软件工程文档（需求规格、设计说明、测试说明）
对应要求	设计一个数据库管理系统，按照软件工程规范方法的要求进行开发（方案 1）
文档版本	v1.0
测试证据	docs/test_results.json，2026-10-28，279 项全部通过
配套文档	设计与测试报告、使用说明、INTERNAL_DESIGN.txt、RESEARCH_DESIGN.txt
生成方式	python tools/build_se_docs.py（测试用例逐条核对真实测试结果）

修订记录

版本	日期	修订内容	修订人
v1.0	2026-10-24	首版：需求分析、UML 设计、测试用例、分工模板	_____

01 / AURORADB

引言

1.1 编写目的

本文档按照软件工程规范，描述 AuroraDB 的需求、总体设计、详细设计和测试设计，作为小组开发、教师验收和答辩的依据。它补充《设计与测试报告》中的实现细节：本文档关注“做什么、怎样分解、怎样验证”，报告关注“具体如何实现、实测结果如何”。

1.2 项目范围

AuroraDB 是一个用 Python 标准库从零实现的单机关系数据库管理系统。作业要求的五个组件（存储管理器、缓冲管理器、查询处理器、执行引擎、用户界面）全部自研实现，不调用 SQLite、MySQL 等现成数据库；SQLite 只在测试中作为结果对照。系统另外实现了事务与崩溃恢复、执行计划展示、一致性检查、AI 助手和研究扩展。

1.3 术语

术语	说明
页（Page）	磁盘与内存之间交换数据的基本单位，固定 4096 字节
RID	记录标识 [页号, 槽号]，跨重启稳定，索引通过它定位堆表记录
堆表（Heap）	按插入顺序存放记录的分槽页集合
B+ 树	内部节点存分隔键、叶节点存 (键, RID) 并用 next 串联的平衡索引
缓冲池 / LRU	内存中的页缓存；容量满时淘汰最久未使用的页
暂存页（Staged page）	事务内被修改的页副本，提交前不写入数据文件
WAL	预写式日志；本系统记录完整页镜像（物理重做日志）
NO-STEAL / FORCE	未提交数据不写盘 / 提交返回前数据页已写盘并同步
AST	抽象语法树，SQL 经语法分析后得到的结构化表示
物理计划	由 TableScan、IndexScan、Filter、HashJoin 等算子组成的执行树
poisoned（中毒）	提交过程发生 I/O 异常、结果不确定时连接所处的状态，必须重开恢复

1.4 参考资料

① 课程大作业方案 1 要求及其数据库管理系统组件构成图；② 本项目 README.md、docs/INTERNAL_DESIGN.txt、docs/RESEARCH_DESIGN.txt；③ SQLite Architecture 与 Atomic Commit 文档；④ PostgreSQL EXPLAIN 文档。

02 / AURORADB

需求分析

需求来源：作业要求的五个组件 + 可验证、可演示、可交付

2.1 涉众分析

涉众	身份	主要关注点	备注
数据库使用者	课程学生、演示者	通过工作台或命令行执行 SQL，查看结果、计划与统计	熟悉基本 SQL
评审教师	课程验收方	检查五个组件是否自研实现、文档与测试是否规范	关注可验证证据
开发小组	3–4 名组员	按模块开发、测试、打包与答辩	见第 07 章分工
学校 AI 服务	外部系统（可选）	根据问题与表结构给出 SQL 建议，不执行 SQL	默认关闭

2.2 用例图

数据库使用者是主要参与者；学校 AI 服务是可选的外部系统参与者，只参与 AI 助手与语义实验。维护数据（UC-03）总是在事务中执行，因此包含（«include»）控制事务（UC-04）；查看执行计划（UC-05）是对执行查询（UC-01）的扩展（«extend»）。

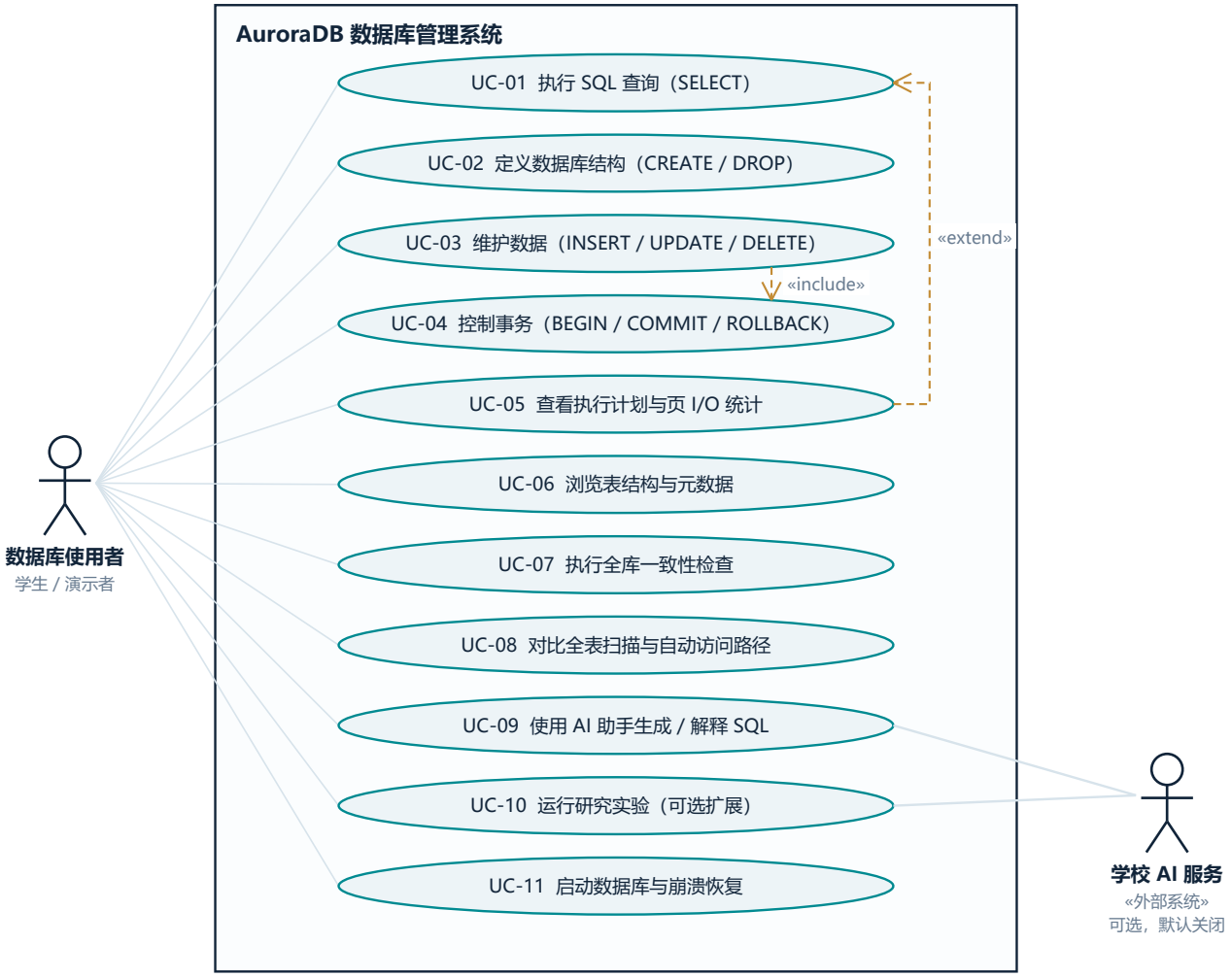


图 2-1 AuroraDB 用例图

2.3 用例清单

编号	参与者	说明	对应需求
UC-01	数据库使用者	执行 SELECT，返回结果、物理计划与 I/O 统计	FR-07–FR-10
UC-02	数据库使用者	CREATE / DROP TABLE，CREATE [UNIQUE] / DROP INDEX	FR-04, FR-11
UC-03	数据库使用者	INSERT / UPDATE / DELETE 并检查约束；包含 UC-04	FR-12
UC-04	数据库使用者	显式事务或单语句自动提交	FR-13, FR-14
UC-05	数据库使用者	EXPLAIN / EXPLAIN ANALYZE；扩展 UC-01	FR-15
UC-06	数据库使用者	SHOW TABLES、DESCRIBE、工作台侧栏	FR-04
UC-07	数据库使用者	CHECK DATABASE 校验页、目录、堆、约束与索引	FR-18
UC-08	数据库使用者	同一 SELECT 强制全表扫描与自动路径对比	FR-19
UC-09	使用者、学校 AI 服务	生成 / 解释 / 优化建议，本地编译校验后由用户执行	FR-20
UC-10	使用者、学校 AI 服务	统计采集、可信改写、预测缓冲、语义算子、自适应连接重排、PPT 现场对比用例	FR-21–FR-23
UC-11	数据库使用者	打开目录、加锁、WAL 恢复、读取系统目录	FR-01, FR-14

2.4 主要用例规约

UC-01 执行 SQL 查询

项目	内容
参与者	数据库使用者
前置条件	数据库已打开；目标表已存在
主事件流	1. 用户在编辑器输入 SELECT 并按 Ctrl+Enter（或在 REPL 中以分号结束）；2. 界面把 SQL 发送给 Database.execute()；3. 系统完成词法、语法分析和名称绑定；4. 规划器选择 IndexScan 或 TableScan，并选择哈希连接或嵌套循环连接；5. 执行引擎完成过滤、连接、分组、排序、投影、LIMIT；6. 界面显示结果表、物理计划树、实际扫描行数与页读写计数
备选 / 异常流	3a. 语法错误：返回含行号、列号的错误信息，不执行；3b. 列名不存在或有歧义、类型不兼容：返回 DatabaseError；3c. 当前处于显式事务：读到本事务尚未提交的修改
后置条件	数据不变；统计计数更新

UC-03 维护数据

项目	内容
参与者	数据库使用者
前置条件	目标表已存在
主事件流	1. 用户提交 INSERT / UPDATE / DELETE；2. 若无活动事务，系统自动开始隐式事务（包含 UC-04）；3. 系统检查类型、NOT NULL、VARCHAR 长度、PRIMARY KEY / UNIQUE；4. 修改堆页并同步维护全部索引，写入事务暂存区；5. 自动提交：先写 WAL 并 fsync，再写数据页；6. 返回受影响行数
备选 / 异常流	3a. 违反约束：整条语句（含多行 INSERT）不产生任何修改，返回错误；5a. 提交 I/O 失败：连接进入 poisoned 状态，须关闭重开后由 WAL 确定结果
后置条件	成功时修改已持久化；失败时数据库保持语句执行前状态

UC-04 控制事务

项目	内容
参与者	数据库使用者
前置条件	数据库已打开
主事件流	1. 用户执行 BEGIN；2. 后续读写在暂存页中进行，查询可读到本事务修改；3. 用户执行 COMMIT，系统按 WAL → 数据文件顺序持久化
备选 / 异常流	3a. 用户执行 ROLLBACK：丢弃暂存页并恢复目录快照；2a. 事务中任一语句出错：系统回滚整个事务并回到自动提交模式（设计取舍，见 5.6）；2b. 关闭连接时仍有活动事务：自动回滚
后置条件	已提交事务满足原子性与持久性；未提交修改不进入 pages.dat

UC-11 启动数据库与崩溃恢复

项目	内容
参与者	数据库使用者
前置条件	指定的数据库目录可读写
主事件流	1. 用户双击启动或执行 python -m auroradb --db 目录；2. 系统对 writer.lock 加独占锁；3. 读取并校验 redo.wal：完整的已提交事务被重放，残缺日志被丢弃；4. 读取页 0 与目录分片，恢复系统目录；5. 进入工作台或 REPL
备选 / 异常流	2a. 目录已被其他连接打开：提示先关闭另一连接；3a. 日志存在提交证据但已损坏：拒绝启动并保留文件供诊断；4a. 页 CRC 不匹配：报告数据损坏，不自动覆盖
后置条件	数据库处于最后一次成功提交后的一致状态

2.5 功能需求

优先级“必须”对应作业明确要求的组件；“应该”是保证数据库可用性的必要能力；“可选”为扩展功能。

编号	需求描述	对应组件	优先级	实现位置
FR-01	页式磁盘存储：数据保存在 4096 字节定长页，页头含魔数、页号、长度与 CRC32	(1) 存储管理器	必须	storage.DiskPager

编号	需求描述	对应组件	优先级	实现位置
FR-02	关系表存储：按表模式保存元组，分槽堆页，RID 跨重启稳定	(1) 存储管理器	必须	storage.HeapTable
FR-03	高效访问：单列 B+ 树索引，支持等值、范围、IN、重复键与唯一约束	(1) 存储管理器	必须	storage.BPlusTree
FR-04	系统目录：表、列、约束、索引元数据跨页持久化；SHOW TABLES / DESCRIBE	(1)(3)	必须	engine.Database
FR-05	缓冲管理：有界 LRU 页缓存，统计命中、未命中、淘汰与读写页数	(2) 缓冲管理器	必须	DiskPager._cache
FR-06	事务暂存：未提交修改只存在于内存暂存页，不写入数据文件（NO-STEAL）	(2) 缓冲管理器	必须	DiskPager._staged
FR-07	SQL 编译：词法与递归下降语法分析，错误定位到行、列	(3) 查询处理器	必须	sql.tokenize / _Parser
FR-08	语义分析：名称绑定、歧义检查、类型检查、GROUP BY 合法性检查	(3) 查询处理器	必须	Database._bind
FR-09	查询计划与优化：索引选择、谓词下推、哈希连接 / 嵌套循环选择	(3) 查询处理器	必须	Database._scan_choice
FR-10	执行查询计划：扫描、过滤、连接、聚合、去重、排序、投影、LIMIT/OFFSET	(4) 执行引擎	必须	Database._select
FR-11	数据定义：CREATE / DROP TABLE，CREATE [UNIQUE] / DROP INDEX	(4) 执行引擎	必须	Database._execute
FR-12	数据操纵与约束：INSERT / UPDATE / DELETE；PK、UNIQUE、NOT NULL、DEFAULT、类型	(4) 执行引擎	必须	Database._write
FR-13	事务控制：BEGIN / COMMIT / ROLLBACK，单条写语句自动提交	(4) 执行引擎	应该	Database._begin/_commit
FR-14	持久性与恢复：页镜像 Redo WAL，崩溃后恢复到最后一次提交	(1)(2)	应该	DiskPager.commit/_recover
FR-15	计划展示：EXPLAIN 与 EXPLAIN ANALYZE，显示估计行数与实际行数	(3)(5)	应该	Database._execute
FR-16	图形用户界面：浏览器 SQL 工作台，显示结果、计划与 I/O 统计	(5) 用户界面	必须	server.py, web/
FR-17	命令行界面：REPL、--execute、--file，错误返回非零退出码	(5) 用户界面	必须	__main__.py
FR-18	一致性检查：CHECK DATABASE 校验页 CRC、目录、堆、约束和索引	扩展	可选	integrity.py
FR-19	扫描对比：同一查询的全表扫描与自动路径实测对比	扩展	可选	Database.compare_query
FR-20	AI 助手：生成、解释、优化 SQL；只做本地编译校验，不自动执行	扩展	可选	ai.py
FR-21	研究实验：统计与代价学习、可信改写、预测缓冲、语义算子	扩展	可选	optimizer / learning / semantic

编号	需求描述	对应组件	优先级	实现位置
FR-22	自适应连接重排：adaptive 模式下对 3–6 张表的内连接用有界动态规划选择连接顺序；置信度、搜索预算或预期收益不足时回退 SQL 书写顺序	扩展	可选	join_optimizer / workload
FR-23	PPT 现场对比用例：在独立临时库中对比 SQL 顺序与自适应规划，核对结果并展示耗时与连接比较次数	扩展	可选	optimizer_demo.py

2.6 非功能需求

编号	类别	需求	验证方法
NFR-01	可靠性	已提交事务原子、持久；页与日志损坏可检测，不静默返回错误数据	真实子进程崩溃测试、故障注入、CRC 篡改测试
NFR-02	正确性	查询结果与成熟数据库一致	与 SQLite 的 180 次差分测试 + 384 次独立查询审计
NFR-03	性能	有选择性的索引查询显著减少扫描记录与页读取	tools/benchmark.py（5000 行基准）
NFR-04	安全	仅监听 127.0.0.1；校验 Host、Origin 与会话令牌；SQL 不作为 Python 执行；API Key 不进入浏览器与提交包	HTTP 安全测试、提交包排除测试
NFR-05	易用性	双击即可启动；中文界面；错误信息带行列号	可执行程序冒烟测试
NFR-06	可移植性	运行时只依赖 Python 标准库；提供 Windows 单文件 exe	在独立目录运行 exe
NFR-07	可维护性	分层模块、单向依赖、统一异常类型、源码注释与自动化回归测试	tools/verify.py
NFR-08	容量约束	单条记录 ≤ 4076 字节页载荷；索引键 ≤ 1024 字节；单连接串行执行	边界测试

2.7 约束与不支持范围

运行环境：Windows 64 位可直接运行 exe；源码运行需要 Python 3.12 或以上。单机单连接，同一数据库目录同时只允许一个连接打开。

当前版本不支持：外键、复合主键、子查询、COUNT(DISTINCT …)、CASE / CAST、窗口函数、RIGHT / FULL JOIN、逗号连接、ALTER TABLE、视图、触发器、用户权限、多会话并发（MVCC）、空间回收（VACUUM）。不支持的语法会显式报错，不会被静默忽略。

03 / AURORADB

概要设计

分层体系结构与模块划分

3.1 设计原则

① 分层：界面 → 查询处理 → 执行 → 记录与索引 → 缓冲 → 页文件，只允许上层调用下层；② 存储层不理解 SQL，SQL 层不直接读写文件；③ 所有持久化修改都经过 DiskPager 的事务接口；④ 不支持的功能显式报错；⑤ 每个模块都有独立的自动化测试。

3.2 体系结构

下图与作业给出的“数据库管理系统组件构成”逐层对应。作业图中的存储管理器在本系统中分为两层：上层 HeapTable / BPlusTree 负责记录和索引的组织，下层 DiskPager 负责页编码、日志与磁盘 I/O；缓冲管理器位于两层之间。

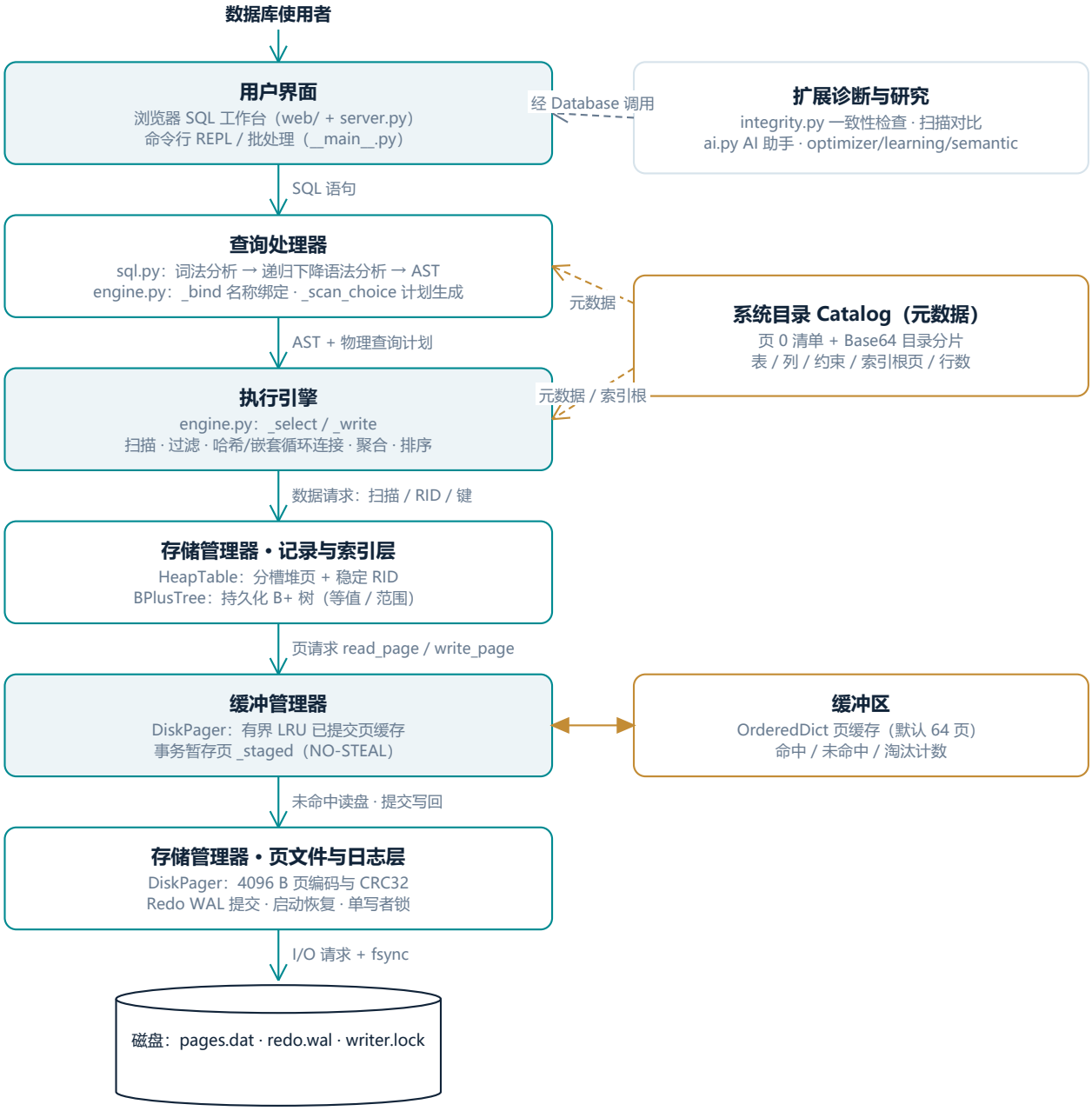


图 3-1 体系结构与作业组件对应关系

3.3 模块划分

模块	源文件	行数	职责	对外接口
SQL 前端	auroradb/sql.py	684	词法分析、递归下降语法分析、AST 深度检查	tokenize(sql)、parse(sql)
查询处理与执行	auroradb/engine.py	1250	目录管理、名称绑定、计划生成、关系算子、约束、事务	Database.execute(sql)
存储与缓冲	auroradb/storage.py	1169	页编码与 CRC、LRU 缓冲、事务暂存、WAL、堆表、B+ 树	DiskPager / HeapTable / BPlusTree
一致性检查	auroradb/integrity.py	473	绕过缓存读取物理页，校验目录、堆、约束与索引	check_database(db)

模块	源文件	行数	职责	对外接口
Web 服务	auroradb/server.py	414	HTTP 路由、请求校验、会话令牌、AI 与研究接口	serve(database, port)
命令行	auroradb/__main__.py	207	参数解析、REPL、批处理、结果表格输出	main(argv)
前端页面	auroradb/web/	1605	编辑器、结果表、计划树、诊断与 AI 面板	HTTP JSON 接口
AI 助手	auroradb/ai.py	556	OpenAI 兼容接口调用、限流、超时、输出清洗	AIService.assist()
研究扩展	optimizer.py / learning.py / semantic.py	1102	可信改写、统计与代价学习、语义算子	optimize() 等
连接优化	join_optimizer.py / workload.py	409	有界动态规划连接重排、去字面量的工作负载画像与反馈	choose_inner_join_order()
PPT 对比用例	auroradb/optimizer_demo.py	126	在独立临时库构建固定数据，交替测量两种连接策略并核对结果	run_demo()

行数为生成本文档时的源码实际行数（含注释与空行）。

3.4 模块接口

接口	提供者	调用者	输入	输出 / 异常
parse(sql)	sql.py	Database	SQL 文本	list[AST dict]；错误抛出 SQLError(line, column)
Database.execute(sql)	engine.py	server.py / __main__.py	一条或多条 SQL	list[Result]；错误抛出 DatabaseError
DiskPager.read_page(id)	storage.py	HeapTable / BPlusTree / Database	页号	页内容 dict（副本）或 None
DiskPager.write_page / allocate	storage.py	HeapTable / BPlusTree / Database	页号、页内容	写入暂存区；allocate 返回新页号
DiskPager.begin / commit / rollback	storage.py	Database	—	提交失败抛出 StorageError 并进入 poisoned
HeapTable.insert / get / update / delete	storage.py	Database	行 dict、RID	RID = [页号, 槽号]
BPlusTree.search / range / insert / remove	storage.py	Database	键、RID、范围边界	list[RID]
check_database(db)	integrity.py	Database.integrity_check	数据库对象	{ok, checks, issues}
choose_inner_join_order(sources, base_rows, ...)	join_optimizer.py	Database._select（adaptive 模式）	连接图、基表行数、基数与代价估计函数	候选顺序、置信度、预期收益；不满足条件时说明回退原因

3.5 Web 接口

方法	路径	请求体	响应
GET	/	—	工作台页面与静态资源
GET	/api/session	—	会话令牌（后续 POST 必须携带 X-Aurora-Token）
GET	/api/schema	—	{tables: [表结构]}
GET	/api/stats	—	缓冲池与 I/O 统计
POST	/api/query	{sql}	{results: [Result]}；SQL 错误返回 400 {error}
POST	/api/integrity	{}	一致性检查报告
POST	/api/compare	{sql}	全表扫描与自动路径的计划、统计与结果一致性
POST	/api/ai	{action, prompt, sql}	AI 建议及本地编译校验结果（不执行）
POST	/api/research/join-demo	{}（固定用例，不接受 SQL）	两种连接策略的结果、耗时中位数、连接比较次数与计划；同一时间只允许运行一次
GET / POST	/api/ai/status、/api/research/*	见 RESEARCH_DESIGN.txt	AI 状态与研究实验

所有 POST 请求必须是 application/json，且携带 /api/session 返回的令牌；Host 只能是 127.0.0.1 或 localhost；拒绝跨来源请求，请求体不超过 512 KiB。

3.6 查询结果数据结构

字段	类型	含义
columns	list[str]	输出列名（别名或表达式文本）
rows	list[list]	结果行；NULL 为 null
row_count	int	结果行数或受影响行数
message	str	如 "1 row(s) inserted"、"Transaction committed"
plan	dict | null	物理计划树：operator / detail / estimated_rows / actual_rows / children
stats	dict	本语句的 disk_reads、cache_hits、rows_scanned、index_lookups、wal_bytes 等增量
elapsed_ms	float	语句执行耗时

3.7 错误处理设计

异常	产生位置	处理方式
SQLError	sql.py 词法 / 语法分析	带行号、列号；Database 转为 DatabaseError；活动事务回滚
DatabaseError	engine.py 绑定、约束、执行	直接返回界面；工作台显示错误，CLI 返回退出码 1
StorageError	storage.py 页、锁、日志	转为 DatabaseError；提交阶段出错时连接进入 poisoned
HTTP 400 / 403 / 413	server.py 请求校验	不执行请求中的 SQL

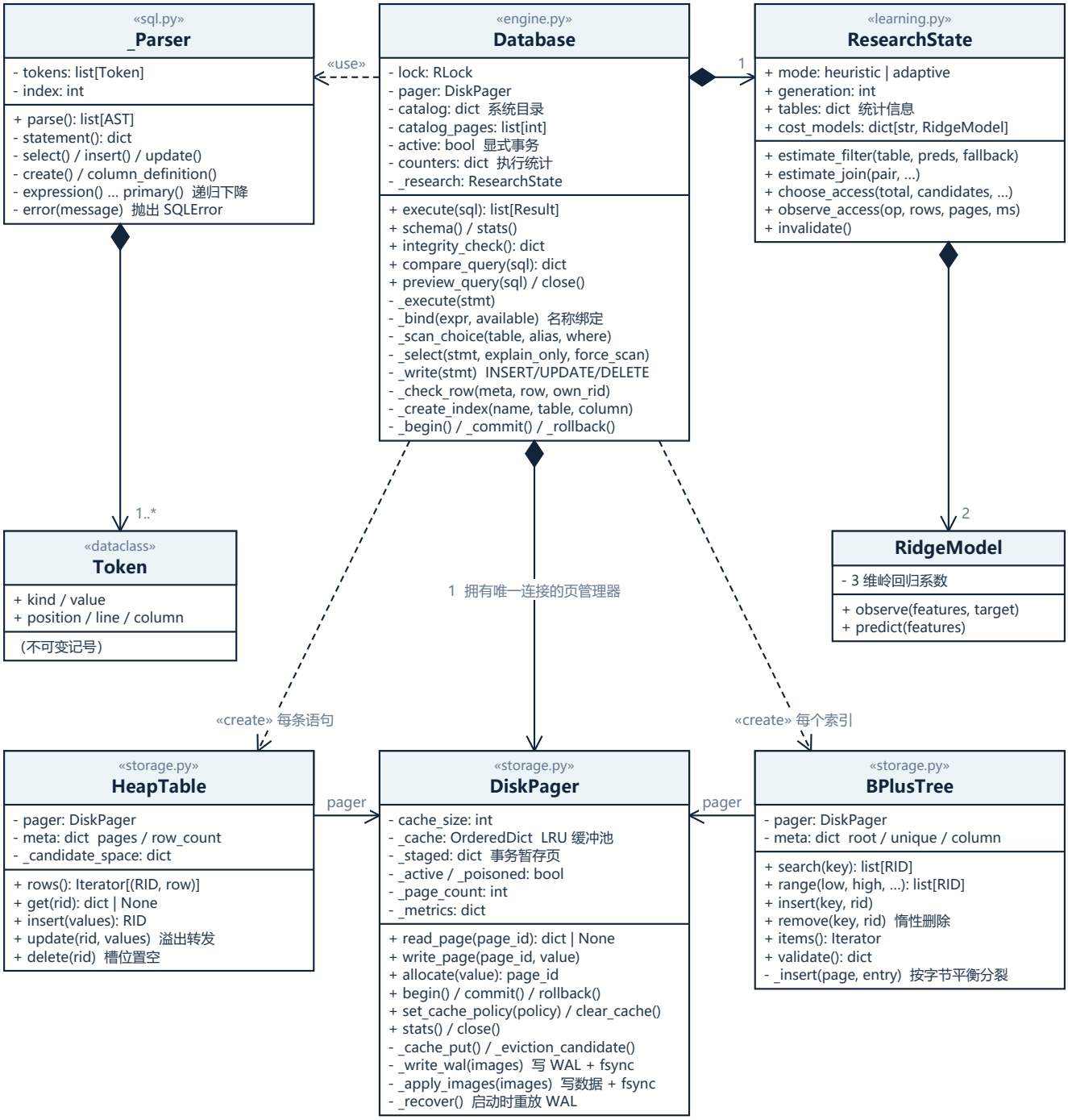
04 / AURORADB

详细设计

UML 类图、时序图、状态图与数据设计

4.1 类图（一）：查询处理、执行与存储

Database 是系统核心：它拥有唯一的 DiskPager，每条语句按需创建 HeapTable 与 BPlusTree 访问对象，两者都通过 DiskPager 读写页，从不直接访问文件。



异常约定：SQLError（sql.py）与 StorageError（storage.py）在 Database.execute() 中统一转换为 DatabaseError，并在事务中自动回滚。

图 4-1 核心类图（engine.py / sql.py / storage.py / learning.py）

4.2 类图（二）：用户界面与扩展服务

Web 服务与命令行是同一个 Database 的两种前端。WorkbenchServer 用 database_lock 串行化请求，多个浏览器标签共享同一个事务会话；AI 请求在网络等待期间不持有数据库锁。

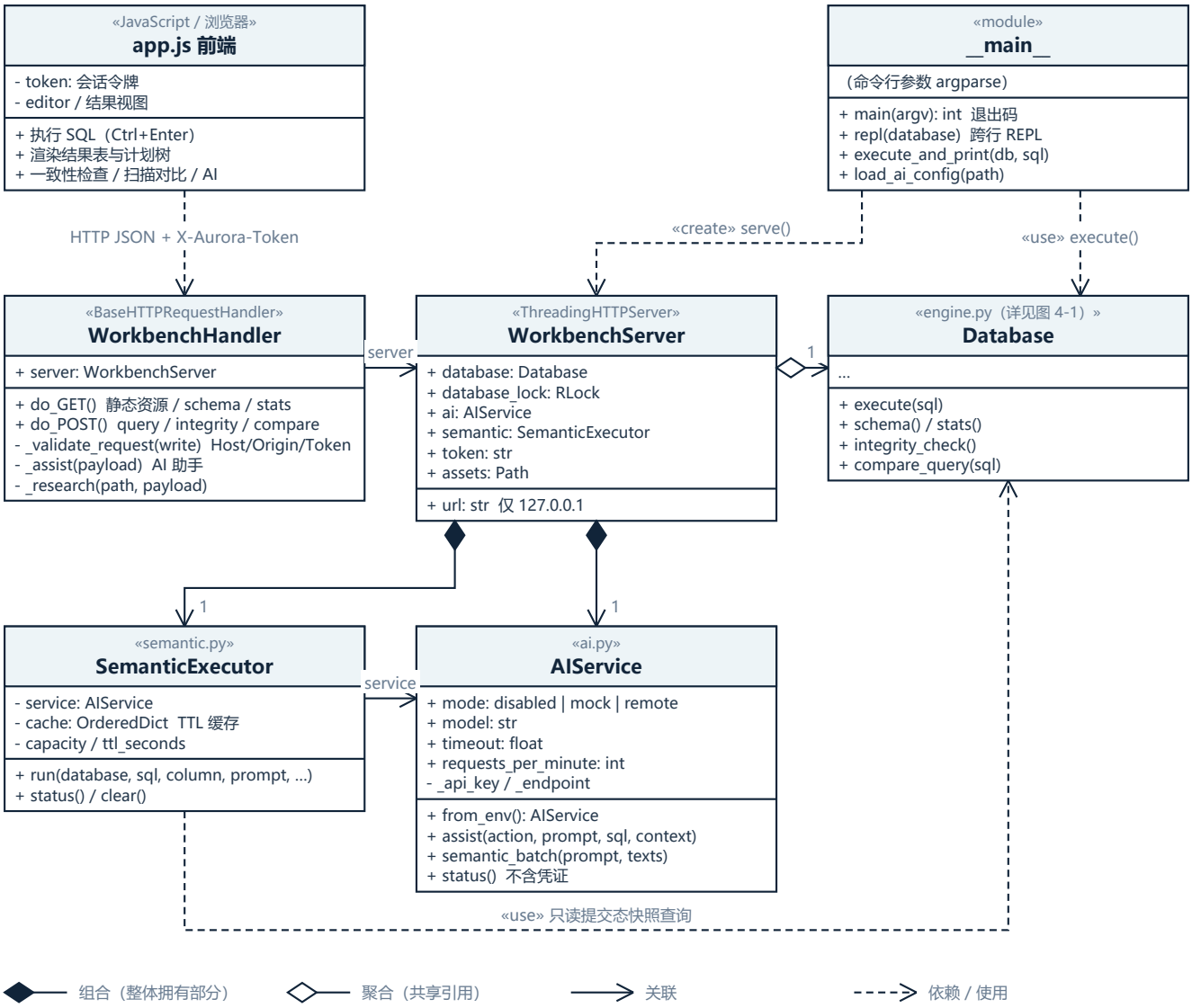


图 4-2 界面层类图（server.py / __main__.py / ai.py / semantic.py）

4.3 主要类职责

类	职责	关键设计
_Parser	把记号序列解析为 AST	每个优先级一个方法；不支持的子句在出错位置报错
Database	目录、绑定、计划、执行、约束与事务	单个 RLock 串行化；事务开始前深拷贝目录用于回滚
DiskPager	页缓冲、事务暂存、WAL 提交与恢复	NO-STEAL + 提交时 FORCE；提交失败即毒化
HeapTable	分槽堆页上的记录增删改查	删除置空槽保持 RID；放大更新转发到溢出页
BPlusTree	单列索引	(键, RID) 复合键支持重复键；按字节平衡分裂；惰性删除
ResearchState	统计信息与代价模型（研究扩展）	默认 heuristic 模式不影响普通查询

类	职责	关键设计
WorkbenchServer / Handler	本机 HTTP 服务	令牌、Host、Origin 校验；JSON 严格解析
AIService	调用学校模型接口	默认关闭；限流、超时、输出清洗；凭证不出后端

4.4 时序图：执行 SELECT 查询（UC-01）

从用户按下 Ctrl+Enter 到结果显示的完整调用链。opt 片段表示只有谓词列上存在索引时才会探测索引；alt 片段表示规划器二选一的访问路径。

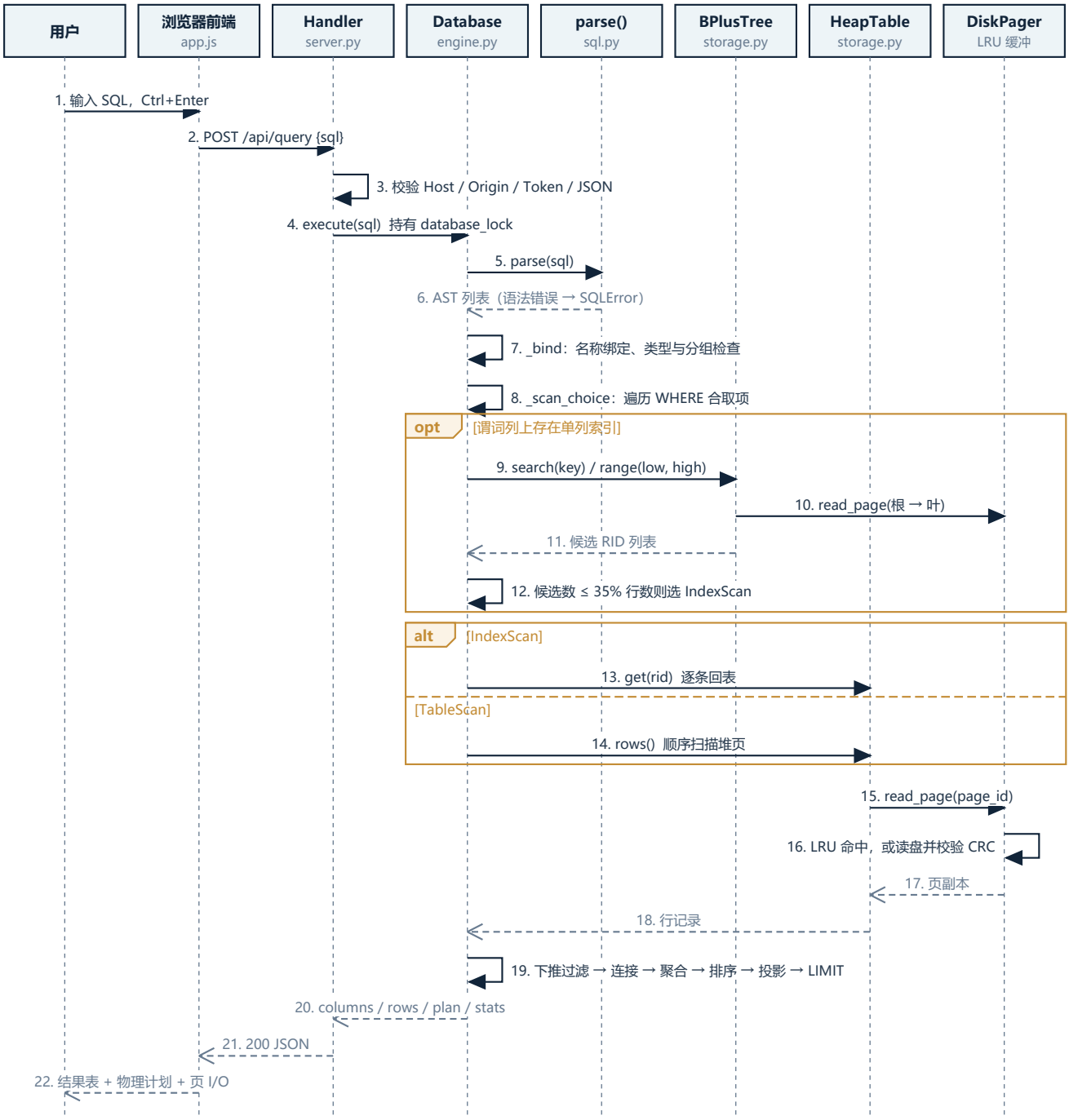


图 4-3 SELECT 查询时序图

4.5 时序图：INSERT 自动提交与 WAL 持久化（UC-03 / UC-04）

所有页修改先进入暂存区；只有全部成功才进入提交协议。提交时先让 WAL 落盘，再写数据文件，因此任何时刻崩溃都能由 WAL 判定事务是否存在。

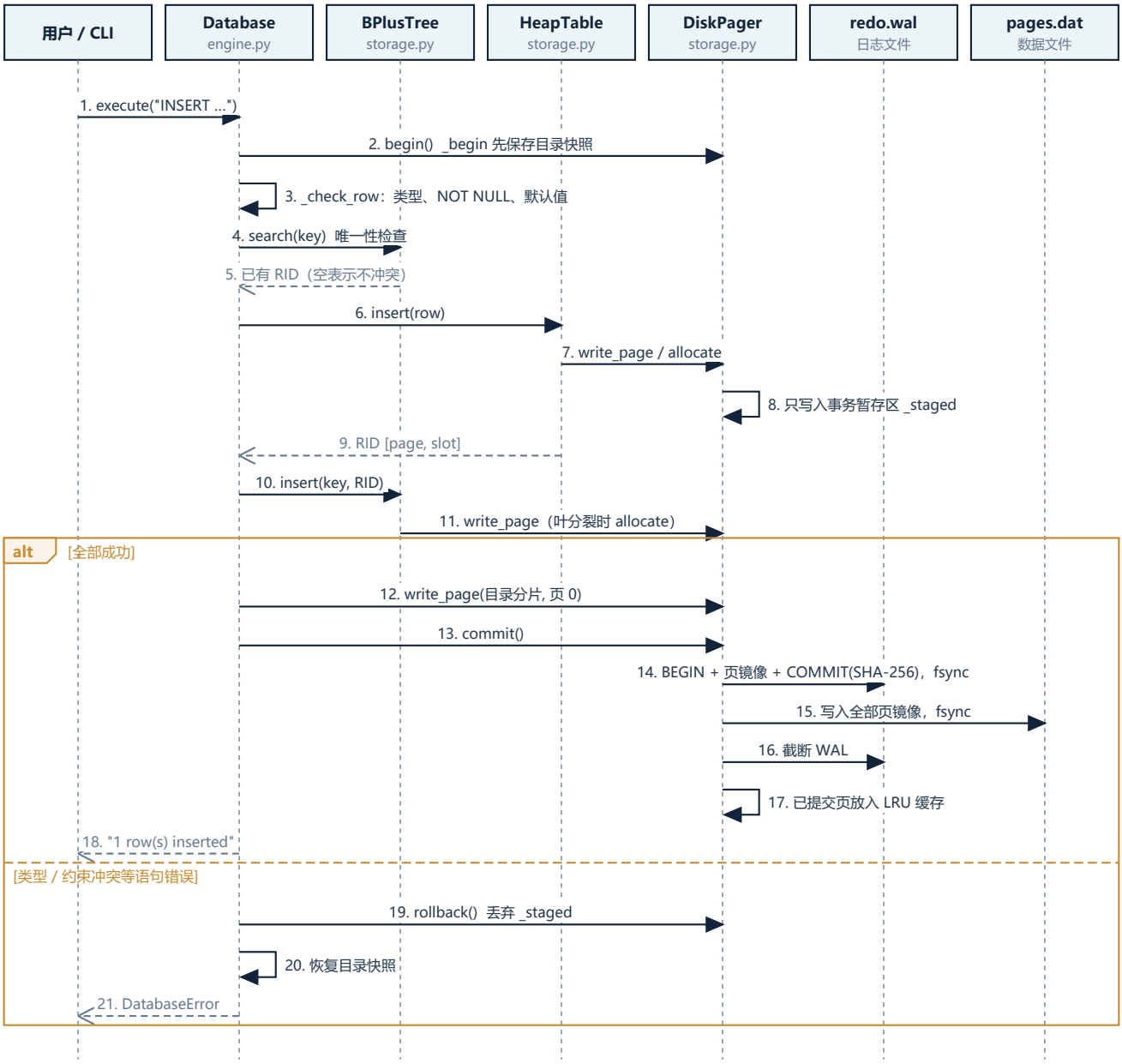


图 4-4 INSERT 自动提交时序图

4.6 时序图：启动与崩溃恢复（UC-11）

恢复只重放通过全部校验的完整事务；无法确定的情况一律拒绝启动并保留现场，不会把可能已提交的事务当作未提交而丢弃。

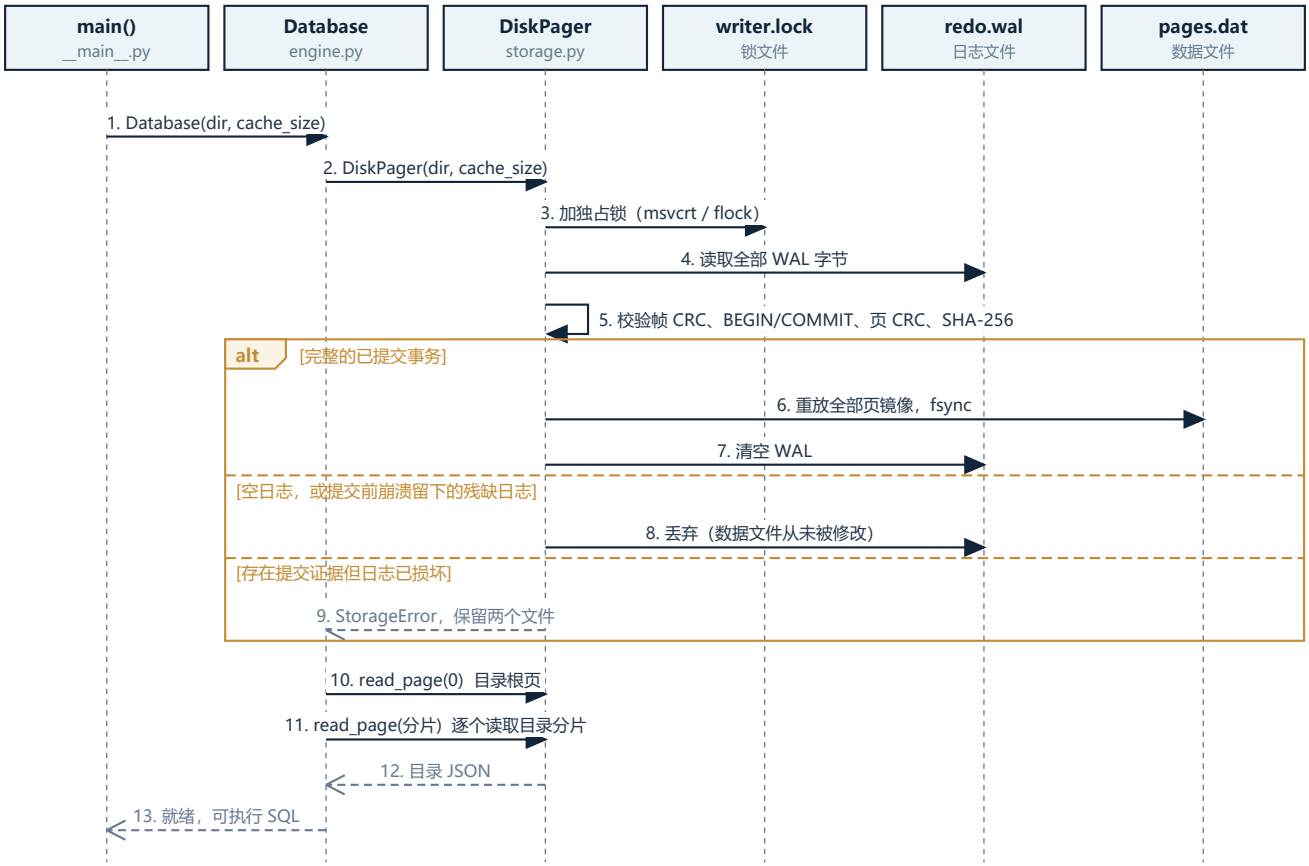


图 4-5 启动与崩溃恢复时序图

4.7 状态图：连接的事务状态

单条写语句在无活动事务时被自动包裹为隐式事务；显式事务中任一语句出错会回滚整个事务并回到自动提交状态（见 5.6 DEF-07）。提交阶段的 I/O 异常使连接进入 poisoned 状态，只能关闭重开。

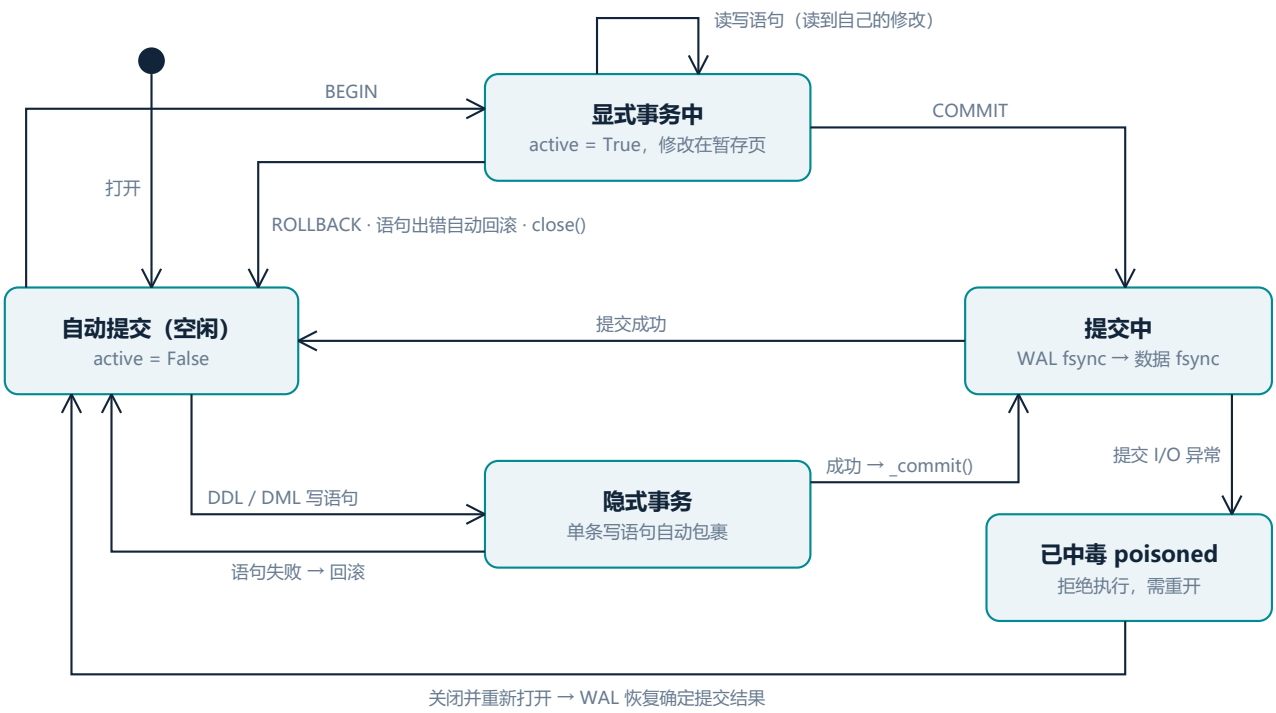

读写语句（读到自己的修改）
BEGIN
显式事务中
active = True，修改在暂存页
COMMIT
打开
ROLLBACK · 语句出错自动回滚 · close()
自动提交（空闲）
active = False
提交成功
提交中
WAL fsync → 数据 fsync
成功 → _commit()
提交 I/O 异常
DDL / DML 写语句
隐式事务
单条写语句自动包裹
已中毒 poisoned
拒绝执行，需重开
语句失败 → 回滚
关闭并重新打开 → WAL 恢复确定提交结果

图 4-6 事务状态图

4.8 数据设计：页格式

偏移	长度（字节）	内容
0	8	魔数 AURPAGE1
8	4	页号（必须与物理偏移一致）
12	4	有效 JSON 字节数
16	4	CRC32（覆盖页号、长度与全部页体）
20	4076	UTF-8 JSON 页体 + 零填充

4.9 数据设计：页内容与系统目录

页体是 UTF-8 JSON 对象，通过 kind 字段区分页类型。系统目录按 2900 字节分片并 Base64 编码，页 0 保存分片清单，所以表的数量不受单页容量限制。

```
页 0（目录清单）  {"kind":"catalog_root","version":1,"pages":[1]}
目录分片          {"kind":"catalog_chunk","ordinal":0,"data":"<Base64>"}
逻辑目录          {"kind":"catalog","version":1,"tables":{"students":{
                  "columns":[...],"pages":[3,7],"row_count":12,
                  "indexes":[{"name":"students_id_key","column":"id","root":2,
  "unique":true}]}}}
堆页              {"kind":"heap","rows":[{"v":{"id":1,"name":"..."}},null,
  {"overflow":9}]}
B+ 树叶节点       {"kind":"bplus_leaf","entries":[[key,[page,slot]],...],"next":12}
B+ 树内部节点     {"kind":"bplus_internal","keys":[[key,[page,slot]],...],"children":[4,
  11]}
```

4.10 数据设计：WAL 格式

部分	内容	作用
文件头	8 字节魔数 AURWAL1\0	标识日志文件
帧头	uint32 长度 + uint32 CRC32	每条记录一个帧
B 记录	提交前页数、目标页数、页镜像数	事务开始
P 记录	页号 + 完整 4096 字节页镜像	物理重做单位
C 记录	目标页数、镜像数 + SHA-256（覆盖 B 与全部 P）	提交标记；缺失则整个事务不重放

正常提交后 WAL 被截断为空；含 N 个页镜像的事务日志大小为 90 + 4113 × N 字节。

4.11 关键算法

算法	说明
索引选择	遍历 WHERE 的合取项，找出“列 op 常量”或“列 IN (常量...)”形式且列上有单列索引的谓词；在索引上取得候选 RID，选候选最少的索引；候选数不超过表行数 35%（至少 1 行）时使用 IndexScan，否则 TableScan。完整 WHERE 条件仍在回表后重新求值，保证结果正确。
谓词下推	只引用基表别名的合取项在读取基表后立即过滤，其余条件在连接之后作为残余过滤执行。
连接算法	ON 条件中存在“左表列 = 右表列”的等值项时，对右表建哈希桶执行哈希连接；否则嵌套循环。LEFT JOIN 对未匹配的左行补 NULL。默认 heuristic 模式按 SQL 书写顺序连接。

算法	说明
连接重排	仅在 adaptive 模式、3–6 张表、全部为 INNER JOIN 且等值连接图连通时启用。用子集动态规划枚举左深连接前缀（候选上限 512，搜索预算 20 ms），以统计信息与学习模型估计中间结果行数和代价，只执行选中的一个顺序。置信度不足、预算或候选超限、预期收益小于不确定性边际时回退 SQL 书写顺序。重排后各表的单表谓词在读取时下推，并可使用索引。工作负载按去掉字面量的查询指纹记录执行反馈；任何写入都会使统计、模型与反馈失效。
B+ 树插入	键与 RID 组成复合键以支持重复键；叶溢出时枚举分裂点，选左右两页 JSON 字节数最接近的位置，右叶最小复合键上推为分隔键；根分裂时分配新根并更新目录中的根页号。删除只移除叶条目（惰性删除）。
LRU 缓冲	OrderedDict 按访问顺序保存已提交页副本；命中移到末尾，超过容量淘汰最久未用页。暂存页独立保存，不参与淘汰，因此未提交数据永远不会写入数据文件。
提交协议	按页号生成全部暂存页的最终镜像 → 写 B/P/C 到 WAL 并 fsync → 写数据页并 fsync → 发布缓存 → 清空 WAL。WAL fsync 之前崩溃，事务不存在；之后崩溃，重启时重放，结果幂等。

05 / AURORADB

测试设计与测试用例

测试证据：2026-10-28T12:37:54

5.1 测试策略

测试层次	测试内容	对应测试
单元测试	词法 / 语法分析、页管理器、LRU、堆表、B+ 树	test_sql、test_storage
集成测试	SQL 层与存储层协同：目录、索引维护、约束、事务回滚	test_engine、test_integration_storage
系统 / 端到端测试	真实 HTTP 服务、命令行子进程、可执行程序	test_server、executable_smoke
差分测试	相同数据和查询在 SQLite 上执行，逐行比对	test_engine（180 次）、audit_queries（384 次）
故障注入与崩溃测试	在提交各持久化边界注入异常或终止进程，检查恢复结果	test_storage、test_integration_storage
回归测试	已修复问题的固定用例：类型、ROUND、AVG 溢出、ORDER BY 别名等	test_regressions
安全测试	Token / Origin / Host、畸形 JSON、密钥不外泄、提交包排除	test_server、test_ai*、test_submission
性能测试	索引与全表扫描的扫描行数、页读取与耗时	tools/benchmark.py

5.2 测试环境与执行方法

项目	内容
操作系统	Windows 11 64 位
运行环境	Python 3.12.14，unittest，仅标准库
执行命令	python tools/verify.py（运行全部测试并写入 docs/test_results.json）
差分审计	python tools/audit_queries.py（写入 docs/audit_queries.json）
性能基准	python tools/benchmark.py（写入 docs/benchmark.json）

5.3 测试用例

自动化用例列出 tests/ 目录下的测试文件与测试方法。生成本文档时，脚本会逐条核对这些方法在测试结果文件中存在且已通过，否则拒绝生成。

编号	模块 / 测试项	前置条件与输入	预期结果	测试实现 / 结果
TC-01	缓冲管理器 事务暂存与回滚隔离	begin → write_page → 读取 → rollback → 再读取	提交前读到暂存版本；回滚后恢复原值，数据文件未被修改	test_storage.py test_no_steal_copy_isolation_and_rollback 结果：通过
TC-02	缓冲管理器 LRU 容量与淘汰	小容量缓存，依次读取超过容量的页，再读最早的页	缓存页数不超过容量；淘汰最久未用页；命中、未命中、淘汰计数正确	test_storage.py test_bounded_lru_cache 结果：通过

编号	模块 / 测试项	前置条件与输入	预期结果	测试实现 / 结果
TC-03	存储管理器 单写者锁	目录已被一个连接打开，另一连接再次打开同一目录	第二个连接被拒绝并提示已被占用	test_storage.py test_writer_lock 结果：通过
TC-04	存储管理器 页 CRC 校验	篡改已提交页的页体、页头和填充字节后读取	抛出 StorageError，不返回损坏数据	test_storage.py test_crc_detects_payload_header_and_padding_damage 结果：通过
TC-05	恢复 提交失败后毒化与恢复	在 WAL fsync 之后注入故障，关闭并重新打开	连接进入 poisoned；重开后按 WAL 恢复已提交事务	test_storage.py test_commit_failure_poison_and_durable_recovery 结果：通过
TC-06	恢复 真实进程崩溃边界	子进程分别在写 WAL 中、WAL fsync 后、写数据页后、数据 fsync 后被终止	WAL fsync 前崩溃：事务不存在；之后崩溃：新页与修改全部存在	test_storage.py test_actual_process_crash_boundaries 结果：通过
TC-07	恢复 撕裂写修复	完整 WAL 已落盘，数据文件尾部出现半页写入	重新打开后重放页镜像，页数与追加页内容正确	test_storage.py test_torn_append_repaired_by_complete_wal 结果：通过
TC-08	恢复 损坏的已提交日志	WAL 含完整提交标记但有记录损坏	拒绝恢复，保留 WAL 与数据文件原字节	test_storage.py test_corrupt_committed_wal_preserves_evidence_and_data 结果：通过
TC-09	存储管理器 堆表增长、溢出转发、删除与重启	插入多页记录 → 放大更新 → 删除 → 重启	RID 保持稳定；转发记录可读；重启后数据一致	test_storage.py test_heap_paging_growth_forwarding_delete_and_reopen 结果：通过
TC-10	存储管理器 删除槽位复用	表中有已删除槽位，继续插入	优先复用空槽；追加插入的读页数线性增长	test_storage.py test_heap_append_reads_scale_linearly_and_deleted_slots_are_reused 结果：通过
TC-11	存储管理器 B+ 树随机重复键、范围、删除、重启	随机插入重复键 → 范围查询 → 删除 → 重启	结果与参照集合一致；validate() 检查通过	test_storage.py test_bplus_random_duplicates_ranges_delete_and_restart 结果：通过
TC-12	存储管理器 B+ 树多层分裂	插入大量变长文本键	树高大于 1；键有序；叶链完整	test_storage.py test_bplus_multilevel_variable_text_keys 结果：通过
TC-13	存储管理器 相同键跨多个叶节点	插入大量相同键后 search(key)	返回全部 RID	test_storage.py test_all_equal_keys_span_linked_leaves 结果：通过

编号	模块 / 测试项	前置条件与输入	预期结果	测试实现 / 结果
TC-14	存储管理器 唯一索引、NULL 与超长键	唯一索引上插入重复键、多个 NULL、超过 1024 字节的键	重复键被拒绝；多个 NULL 允许；超长键被拒绝	test_storage.py test_unique_null_and_oversize_keys 结果：通过
TC-15	查询处理器 注释、引号与位置	含行注释、嵌套块注释、转义引号和多种引号标识符的 SQL	记号类型、值与行列号正确	test_sql.py test_comments_quotes_and_locations 结果：通过
TC-16	查询处理器 词法错误定位	未闭合字符串、非法字符	SQLError 给出准确行号与列号	test_sql.py test_lexical_errors_are_positioned 结果：通过
TC-17	查询处理器 运算符优先级与结合性	1 + 2 * 3 - 4 等算术表达式	AST 结构符合优先级与左结合	test_sql.py test_arithmetic_precedence_and_associativity 结果：通过
TC-18	查询处理器 SELECT 完整子句	带 JOIN、GROUP BY、HAVING、ORDER BY、LIMIT 的查询	各子句的 AST 正确	test_sql.py test_select_joins_group_order_and_limit 结果：通过
TC-19	查询处理器 不支持的语法显式报错	子查询、RIGHT JOIN、ALTER TABLE 等	返回错误，不静默忽略	test_sql.py test_unsupported_or_malformed_sql_is_not_silently_ignored 结果：通过
TC-20	查询处理器 嵌套过深防护	嵌套深度超过 128 层的表达式	返回 SQLError，不发生栈溢出	test_sql.py test_excessive_nesting_is_a_sql_error 结果：通过
TC-21	执行引擎 增删改查与持久化	建表插入 → UPDATE / DELETE → 关闭重开 → SELECT	结果正确，重启后保持不变	test_engine.py test_crud_and_persistence 结果：通过
TC-22	执行引擎 默认值、类型与约束	省略列插入；插入重复主键、NULL 到 NOT NULL 列、错误类型	默认值生效；违规语句报错且行数不变	test_engine.py test_defaults_types_and_constraints 结果：通过
TC-23	执行引擎 多行 INSERT 原子性	第二行违反主键约束的多行 INSERT	整条语句不写入任何行	test_engine.py test_multirow_insert_atomic 结果：通过
TC-24	执行引擎 显式事务与失败写入	BEGIN → UPDATE → ROLLBACK；BEGIN → DELETE → COMMIT；事务中插入重复主键	回滚恢复原值；提交生效；出错后整个事务回滚	test_engine.py test_explicit_transaction_and_failed_write 结果：通过

编号	模块 / 测试项	前置条件与输入	预期结果	测试实现 / 结果
TC-25	执行引擎 关闭连接自动回滚	BEGIN; DELETE FROM people; 后直接关闭并重开	删除未生效	test_engine.py test_close_rolls_back 结果：通过
TC-26	执行引擎 NULL 三值逻辑	score = NULL、IS NULL、NOT (score > 85)、NOT IN (1, NULL)	结果符合 SQL 三值逻辑	test_engine.py test_null_three_valued_logic 结果：通过
TC-27	执行引擎 连接与歧义列	LEFT JOIN、INNER JOIN + WHERE、自连接中引用未限定列	连接结果正确；歧义列报错	test_engine.py test_join_left_join_and_ambiguity 结果：通过
TC-28	执行引擎 聚合、去重、排序、分页	GROUP BY + HAVING、DISTINCT + OFFSET、ORDER BY 列序号、空表聚合	结果正确；非法分组报错；空表聚合返回 0 / NULL	test_engine.py test_aggregate_distinct_sort_limit 结果：通过
TC-29	查询处理器 索引选择与索引维护	250 行表，EXPLAIN ANALYZE 主键等值查询；更新、删除后再查；重启	计划为 IndexScan，rows_scanned = 1；索引随修改正确维护	test_engine.py test_index_maintenance_and_actual_scan 结果：通过
TC-30	查询处理器 EXPLAIN 不执行	EXPLAIN SELECT …	只返回计划，rows_scanned = 0，数据不变	test_engine.py test_explain_does_not_execute_or_mutate 结果：通过
TC-31	执行引擎 与 SQLite 差分测试	固定种子生成 180 轮随机查询与修改，同时在 SQLite 上执行	每轮结果与 SQLite 完全一致	test_engine.py test_seeded_differential_queries_and_mutations 结果：通过
TC-32	存储管理器 系统目录跨页扩展	创建 75 张表 → 回滚 → 重启	目录跨多页保存且与提交状态一致	test_integration_storage.py test_catalog_scales_to_75_tables_and_survives_rollback_restart 结果：通过
TC-33	恢复 进程崩溃后整体原子恢复	写入过程中终止进程，重启	目录、堆表与索引同时恢复到一致状态	test_integration_storage.py test_process_crash_atomically_recovers_catalog_heap_and_index 结果：通过
TC-34	一致性检查 检测页 CRC 损坏	篡改已在缓存中的页后执行检查	报告 CRC 错误	test_integrity.py test_crc_damage_is_detected_even_when_page_is_warm_in_cache 结果：通过
TC-35	一致性检查 检测缺失的索引项	删除一个索引项（页 CRC 仍有效）后检查	报告索引与堆表不一致	test_integrity.py test_missing_index_entry_with_valid_page_crc_is_reported 结果：通过

编号	模块 / 测试项	前置条件与输入	预期结果	测试实现 / 结果
TC-36	诊断 扫描对比	对同一 SELECT 执行 compare_query	两条路径结果一致，文件与目录不变	test_diagnostics.py test_real_scan_comparison_preserves_files_catalog_and_results 结果：通过
TC-37	用户界面 页面、表结构、统计与会话	启动工作台，请求 /、/api/schema、/api/stats、/api/session	返回 200 与正确内容	test_server.py test_static_schema_stats_and_session 结果：通过
TC-38	用户界面 查询与 SQL 错误	POST /api/query 分别提交正确与错误的 SQL	正确返回结果；错误返回 400 与错误信息	test_server.py test_query_and_sql_error 结果：通过
TC-39	安全 令牌、来源与主机校验	缺少令牌、跨来源、错误 Host 的请求	返回 403，不执行 SQL	test_server.py test_token_origin_host_and_route_protection 结果：通过
TC-40	安全 畸形 JSON 不执行	重复字段、NaN、孤立代理字符	返回 400，不执行 SQL	test_server.py test_ambiguous_nonfinite_and_invalid_unicode_json_never_executes 结果：通过
TC-41	用户界面 示例数据幂等与退出码	两次执行 --demo；执行引用不存在表的 SQL	示例只创建一次（12 名学生）；错误时退出码为 1	test_server.py test_demo_is_idempotent_and_batch_exit_code 结果：通过
TC-42	用户界面 REPL 语句结束判断	字符串或注释中包含分号的跨行输入	只在真正的语句结尾执行	test_server.py test_repl_terminators_handle_strings_and_comments 结果：通过
TC-43	AI 助手 未配置时默认关闭	无配置或配置错误时启动	AI 为 disabled，数据库功能不受影响	test_ai.py test_disabled_default_and_bad_configuration_do_not_break_startup 结果：通过
TC-44	AI 助手 建议只编译不执行	远程模型返回生成的 SQL	本地编译校验，不执行、不修改数据	test_ai_integration.py test_remote_generation_is_compiled_locally_and_never_executed 结果：通过
TC-45	交付 提交包排除私有配置	项目中存在 *.local.json 与 .env 文件时打包	这些文件不进入压缩包	test_submission.py test_local_and_environment_files_omitted 结果：通过

编号	模块 / 测试项	前置条件与输入	预期结果	测试实现 / 结果
TC-46	连接优化 动态规划选择连接顺序	large / middle / small 三表连接图，给定基数与代价估计函数	选出 middle → small → large；只调用估计函数，不执行查询	test_join_optimizer.py test_dp_reorders_without_executing_candidates_and_honors_cost 结果：通过
TC-47	连接优化 适用范围限制	含 LEFT JOIN 的三表连接；7 张表的内连接	判定为不适用，并分别说明不支持的连接类型与超出表数上限	test_join_optimizer.py test_rejects_outer_joins_disconnected_graphs_and_relation_limit 结果：通过
TC-48	连接优化 保守回退	基表置信度低；搜索预算为 0；候选上限为 1；预期收益小于不确定性边际	均不采纳新顺序，回退 SQL 书写顺序并给出原因	test_join_optimizer.py test_low_confidence_budget_and_small_benefit_fall_back 结果：通过
TC-49	连接优化 工作负载指纹	仅字面量值不同的两条查询；容量为 2 时记录 3 个模板	指纹相同（字面量类型不同则不同）；模板数不超过容量	test_join_optimizer.py test_workload_fingerprint_masks_literal_values_and_is_bounded 结果：通过
TC-50	连接优化 三表集成执行与失效	2000 / 500 / 1 行三表，采集统计后在 adaptive 模式执行三表连接，再插入一行	预览不读取数据；执行顺序为 m → s → l，结果与 heuristic 模式一致；插入后统计与模型失效并回退	test_join_optimizer.py test_integrated_three_table_plan_preserves_bag_and_reports_feedback 结果：通过
TC-51	研究实验 PPT 用例隔离运行	用户库中已有同名 people 表且有未提交事务时调用 /api/research/join-demo	两种策略均返回 100 行，连接比较 20000 次对 200 次；临时库被清理；用户数据、事务、统计与计数器不变	test_optimizer_demo.py test_live_example_preserves_user_data_transaction_models_and_counters 结果：通过
TC-52	安全 PPT 用例固定接口	缺少令牌；请求体携带 SQL；上一次运行尚未结束时再次请求	分别返回 403、400、429，不执行任何传入的 SQL	test_optimizer_demo.py test_fixed_route_rejects_injection_missing_token_and_duplicate_runs 结果：通过
TC-53	用户界面 页面展示的 SQL 与实际执行一致	读取工作台页面中展示的用例 SQL	与后端实际执行的 SQL 完全相同	test_optimizer_demo.py test_displayed_sql_matches_executed_case 结果：通过
TC-54	系统测试 可执行程序独立运行	在没有 Python 的独立目录运行 dist/AuroraDB.exe（--demo、--execute、--web）	示例库创建成功，查询正确，进程正常退出	docs/executable_smoke.json 结果：通过
TC-55	性能测试 索引减少扫描	5000 行表，冷缓存，SELECT id,reading FROM measurements WHERE code=3210;	IndexScan 扫描的记录数显著少于 TableScan	docs/benchmark.json 结果：通过（扫描 5000 行 → 1 行）

5.4 需求追踪矩阵

每条功能需求都至少由一个测试用例覆盖；“扩展”类需求另由对应测试文件整体覆盖。

需求	名称	组件	覆盖的测试用例
FR-01	页式磁盘存储	(1) 存储管理器	TC-03、TC-04
FR-02	关系表存储	(1) 存储管理器	TC-09、TC-10
FR-03	高效访问	(1) 存储管理器	TC-11、TC-12、TC-13、TC-14、TC-29、TC-55
FR-04	系统目录	(1)(3)	TC-32
FR-05	缓冲管理	(2) 缓冲管理器	TC-02
FR-06	事务暂存	(2) 缓冲管理器	TC-01
FR-07	SQL 编译	(3) 查询处理器	TC-15、TC-16、TC-17、TC-18、TC-19、TC-20
FR-08	语义分析	(3) 查询处理器	（由扩展测试文件覆盖）
FR-09	查询计划与优化	(3) 查询处理器	TC-29、TC-55
FR-10	执行查询计划	(4) 执行引擎	TC-21、TC-26、TC-27、TC-28、TC-31
FR-11	数据定义	(4) 执行引擎	TC-21
FR-12	数据操纵与约束	(4) 执行引擎	TC-14、TC-21、TC-22、TC-23
FR-13	事务控制	(4) 执行引擎	TC-01、TC-23、TC-24、TC-25
FR-14	持久性与恢复	(1)(2)	TC-05、TC-06、TC-07、TC-08、TC-33
FR-15	计划展示	(3)(5)	TC-30
FR-16	图形用户界面	(5) 用户界面	TC-37、TC-38、TC-54
FR-17	命令行界面	(5) 用户界面	TC-41、TC-42、TC-54
FR-18	一致性检查	扩展	TC-34、TC-35
FR-19	扫描对比	扩展	TC-36
FR-20	AI 助手	扩展	TC-43、TC-44
FR-21	研究实验	扩展	（由扩展测试文件覆盖）
FR-22	自适应连接重排	扩展	TC-46、TC-47、TC-48、TC-49、TC-50
FR-23	PPT 现场对比用例	扩展	TC-51、TC-52、TC-53
NFR-01	可靠性	非功能	TC-04、TC-06、TC-08、TC-33
NFR-02	正确性	非功能	TC-31、TC-50
NFR-03	性能	非功能	TC-55
NFR-04	安全	非功能	TC-20、TC-39、TC-40、TC-44、TC-45、TC-52
NFR-05	易用性	非功能	（见验证方法）

需求	名称	组件	覆盖的测试用例
NFR-06	可移植性	非功能	TC-54
NFR-07	可维护性	非功能	（见验证方法）
NFR-08	容量约束	非功能	（见验证方法）

5.5 测试结果汇总

测试文件	覆盖内容	测试数	结果
test_advanced_server	HTTP 扩展接口与安全	9	全部通过
test_ai	AI 服务配置、限流与输出清洗	22	全部通过
test_ai_deadlines	AI 网络超时边界	9	全部通过
test_ai_integration	AI 与数据库的 HTTP 集成	8	全部通过
test_chinook_dataset	Chinook 公开数据集导入	1	全部通过
test_diagnostics	扫描对比与诊断	6	全部通过
test_engine	执行引擎、约束、事务、SQLite 差分	16	全部通过
test_integration_storage	SQL 层与存储层集成、崩溃恢复	10	全部通过
test_integrity	全库一致性检查	20	全部通过
test_join_forms	—	15	全部通过
test_join_optimizer	自适应连接重排与工作负载反馈	6	全部通过
test_optimizer_demo	PPT 现场对比用例与接口安全	3	全部通过
test_predictive_buffer	预测缓冲策略	12	全部通过
test_regressions	语义回归用例	18	全部通过
test_research_audit	研究功能交互审计	12	全部通过
test_research_optimizer	可信优化与代价学习	29	全部通过
test_semantic	语义算子	17	全部通过
test_server	HTTP 服务与命令行	13	全部通过
test_sql	SQL 词法与语法分析	23	全部通过
test_storage	页管理器、LRU、WAL、堆表、B+ 树	25	全部通过
test_submission	提交包打包规则	5	全部通过

合计 279 项自动化测试，失败 0 项，错误 0 项，耗时约 116 秒。另有独立查询审计 384 次（有无索引结果一致性 + SQLite 连接 / 分组对照），不一致 0 次。

5.6 已知缺陷与改进计划

以下问题由 2026-10-24 的代码审查与 2 万行数据实测发现（Windows 11，Python 3.12）。它们不影响查询结果的正确性，主要影响性能或与标准 SQL 的一致程度。修复后请同步更新本表。

编号	级别	问题	实测现象	位置	改进方案
DEF-01	中	UPDATE / DELETE 的 WHERE 不使用索引，即使按主键定位也会全表扫描	2 万行表：主键点查 2.5 ms，按主键 UPDATE 220 ms	engine.py _write	复用 _scan_choice 选出目标 RID
DEF-02	中	CREATE INDEX 逐行插入 B+ 树，每次插入都要重新序列化整页	2 万行表建索引约 28 s	engine.py _create_index	先按 (key, RID) 排序，再自底向上批量构建
DEF-03	中	固定 35% 阈值未考虑回表代价，中等选择率下索引路径可能比全表扫描慢	选择率 10%：TableScan 194 ms，IndexScan 480 ms	engine.py _scan_choice	引入页数代价估计，或把 RID 按页排序后回表
DEF-04	低	默认 heuristic 模式与两表连接中，被连接表总是全表扫描，WHERE 中只涉及该表的条件不会下推；adaptive 模式下 3–6 表内连接被重排时已下推并可用索引	两表 JOIN … WHERE s.id = 5 仍扫描 2 万行	engine.py _select	把重排路径中的谓词下推推广到所有 INNER JOIN
DEF-05	低	删除与 DROP 后的页不回收，B+ 树空叶不合并	文件只增不减	storage.py	增加空闲页链表或 VACUUM
DEF-06	低	SELECT DISTINCT 的 ORDER BY 可引用非输出列，标准 SQL 应拒绝	—	engine.py _select	绑定阶段增加检查
DEF-07	设计取舍	显式事务中任一语句出错即回滚整个事务并退出事务模式（PostgreSQL 会保留已中止状态，等待用户 ROLLBACK）	由 TC-24 固定该行为	engine.py execute	答辩时说明理由；或改为“已中止”状态

06 / AURORADB

项目管理与配置管理

6.1 开发过程模型

项目采用迭代增量模型：先完成可独立测试的存储内核，再逐层向上实现 SQL 与界面，每个迭代都以对应测试全部通过作为完成标准。实际起止日期请小组按真实情况填写。

迭代	主题	主要产出	完成标准	实际起止日期
迭代 1	存储内核	页格式与 CRC、DiskPager、LRU、堆表、单写者锁	test_storage 页与堆表用例通过	_____
迭代 2	索引与恢复	B+ 树、WAL 提交协议、启动恢复、崩溃测试	崩溃边界与 B+ 树用例通过	_____
迭代 3	SQL 前端与执行	词法 / 语法、绑定、计划、关系算子、约束、事务	test_sql、test_engine、SQLite 差分通过	_____
迭代 4	界面与交付	Web 工作台、REPL、exe 打包、使用说明	HTTP / CLI 测试与 exe 冒烟测试通过	_____
迭代 5	扩展与文档	一致性检查、扫描对比、AI 助手、研究实验、报告	扩展功能测试与报告证据通过	_____
迭代 6	数据集与连接优化	Chinook 公开数据集、自适应连接重排、PPT 现场用例、Mac 启动与自检	全部自动化测试通过	_____

6.2 配置管理

项目	内容
版本控制	Git；远程仓库 https://github.com/ipromise77/AuroraDB，主分支 main
版本号	pyproject.toml 与 auroradb/__init__.py 统一维护，当前 1.2.0
忽略规则	.gitignore 排除数据库文件、日志、构建目录与 *.local.json 私有配置
构建	python tools/build_exe.py 生成 dist/AuroraDB.exe
回归	每次提交前运行 python tools/verify.py，全部通过才合并
交付	python tools/package_submission.py 生成提交包，MANIFEST.json 记录每个文件的 SHA-256

6.3 版本记录

提交	日期	作者	说明
7d40b6c	2026-10-28	HuangJZh	feat(optimizer): extend join optimization and update defense materials
cc522c1	2026-10-28	yuankai	docs(presentation): clarify storage and buffer module with updated results
a0df6c0	2026-10-28	yuankai	perf(storage): bypass low-reuse scan pages and avoid redundant copies
60af7c0	2026-10-27	GP	presentation and docs: drop two footnotes, 259-test evidence
2297197	2026-10-27	GP	presentation: 黄俊植's updated section 04, guide matches the workbench

提交	日期	作者	说明
c09ce24	2026-10-27	GP	presentation guide: WPS presenter view and fonts to install on the Mac
5c65581	2026-10-27	GP	presentation: name _parse_semantic_result() in the AIService diagram
e18606e	2026-10-27	GP	presentation: add section 05, full five-section deck
9549440	2026-10-27	lumingchun 123	refactor: separate semantic requests from response parsing
6917983	2026-10-27	GP	presentation: add section 04, template palette for sections 01-03
91124b5	2026-10-27	GP	presentation: merge section 03 into the group deck, larger type
6b8737b	2026-10-27	GP	feat(presentation): Mac demo kit, warm workbench theme, trusted-rewrite gates
6a99aba	2026-10-26	HuangJZh	feat(workbench): add PPT optimizer demo and macOS launcher
c28d197	2026-10-25	HuangJZh	Merge branch 'main' of https://github.com/ipromise77/AuroraDB
cd77c4c	2026-10-25	HuangJZh	feat(optimizer): add workload-driven adaptive join planning
8cad86d	2026-10-25	GP	docs: add Mac Codex handoff guide
2017f7f	2026-10-25	GP	build: validate Chinook evidence in submission package
cfd36cd	2026-10-25	GP	test: refresh final evidence after Chinook update
321b4de	2026-10-25	GP	docs: regenerate Chinook-aware reports and evidence
54bc93b	2026-10-25	GP	docs: refresh dataset citation note
fae8c12	2026-10-25	GP	docs: document Chinook dataset mode
37ef646	2026-10-25	GP	feat: add Chinook public dataset mode
b663810	2026-10-24	GP	Fix CJK terminal commands in defense guide
d0fd537	2026-10-24	GP	Normalize defense guide QA evidence
976a044	2026-10-24	GP	Add four-person defense runbook PDF
d411594	2026-10-24	GP	Split project into four components with one innovation each
8b5215d	2026-10-24	GP	Balance four-member defense responsibilities
aaa6009	2026-10-24	GP	Document four-member project responsibility plan
b00c69e	2026-10-23	GP	Integrate SJTU campus AI API and harden validation
644b5eb	2026-10-12	GP	Initial AuroraDB v1.2.0 course project with verified research extensions

建议：今后每位组员用自己的 Git 账号提交所负责模块的修改，使提交记录能够反映实际分工。

6.4 编码与提交规范

规范	约定
模块注释	每个源文件以文档字符串说明职责、边界与不变量

规范	约定
异常	用户可见错误统一为 DatabaseError；存储层错误为 StorageError
测试命名	test_<被验证的行为>，一个测试方法验证一个行为
依赖	运行时只使用 Python 标准库；文档与打包工具依赖不进入运行时
提交说明	一句话说明改动目的，例如“engine: UPDATE/DELETE 使用索引定位目标行”

07 / AURORADB

小组分工与完成情况

内容来自 docs/分工说明填写表.txt

7.1 子系统与模块对应

子系统	对应作业组件	源文件	对应测试	创新点
数据存储与缓冲	(1) 存储管理器、(2) 缓冲管理器	storage.py、integrity.py	test_storage、test_integration_storage、test_integrity、test_predictive_buffer	扫描感知的预测缓冲策略
SQL 编译与查询执行	(3) 查询处理器、(4) 执行引擎	sql.py、engine.py、optimizer.py	test_sql、test_engine、test_regressions、test_diagnostics	可信查询改写与结果核验
统计学习与性能优化	(3) 查询优化	learning.py、join_optimizer.py、workload.py、optimizer_demo.py	test_research_optimizer、test_join_optimizer、test_optimizer_demo、test_research_audit	域外回退的轻量学习型优化、自适应连接重排
AI 语义与交互交付	(5) 用户界面、可执行程序	semantic.py、ai.py、server.py、__main__.py、web/	test_semantic、test_server、test_advanced_server、test_ai*、test_submission	受预算约束的 AI 语义算子

7.2 成员完成情况

成员 1

项目	内容
姓名	钟国鹏
学号	（待填写）
角色	SQL 编译与可信查询执行
负责模块	SQL 词法/语法、语义绑定、扫描/连接/聚合/事务执行；创新点为可信查询改写、规则证据、候选结果核验和保守推荐
负责源文件	auroradb/sql.py、auroradb/engine.py、auroradb/optimizer.py、tools/optimizer_experiment.py
对应测试	tests/test_sql.py、tests/test_engine.py、tests/test_regressions.py、tests/test_diagnostics.py、tests/test_research_optimizer.py、docs/optimizer_research.json
实际完成内容	拟负责 SQL 编译与可信查询执行组件；与全组共同参与总体架构、其他创新方向接口讨论、需求分析、构思讨论、代码核验、测试复现、报告与答辩材料；提交前由本人确认实际完成内容。
代表性提交	（待填写）
工作量占比（%）	25
本人签名	（待填写）

成员 2

项目	内容
姓名	曾子恒
学号	（待填写）
角色	数据存储与预测缓冲
负责模块	存储管理器（页格式与 CRC、HeapTable、BPlusTree、WAL 与崩溃恢复）、LRU/预测缓冲策略与一致性检查；创新点为扫描感知预测淘汰、无证据回退 LRU
负责源文件	auroradb/storage.py、auroradb/integrity.py、tools/buffer_experiment.py
对应测试	tests/test_storage.py、tests/test_integration_storage.py、tests/test_integrity.py、tests/test_predictive_buffer.py、docs/buffer_research.json
实际完成内容	拟负责数据存储与预测缓冲组件的设计核验和实验复现；与全组共同参与需求分析、构思讨论、接口联调、代码核验、测试复现、报告与答辩材料；提交前由本人确认实际完成内容。
代表性提交	（待填写）
工作量占比（%）	25
本人签名	（待填写）

成员 3

项目	内容
姓名	黄俊植
学号	（待填写）
角色	统计学习与性能优化
负责模块	统计信息、NULL/NDV/直方图、基数估计、访问成本模型、训练/留出查询和差分评估；创新点为域外回退的轻量学习型优化
负责源文件	auroradb/learning.py、auroradb/workload.py、auroradb/join_optimizer.py、auroradb/engine.py（查询规划集成）、tools/audit_queries.py、tools/join_optimizer_experiment.py、tests/test_research_optimizer.py、tests/test_join_optimizer.py、docs/RESEARCH_DESIGN.txt
对应测试	tests/test_research_optimizer.py、tests/test_join_optimizer.py、tests/test_research_audit.py、tools/join_optimizer_experiment.py
实际完成内容	实现列统计与轻量基数/读成本/连接成本模型、字面量类型归一化的有界工作负载反馈、最多6表的INNER JOIN动态规划、置信度/域外/预算回退、执行计划诊断和训练/留出/漂移实验；与全组共同参与需求分析、构思讨论、可信改写与存储缓冲接口评审、差分测试、代码核验、报告与答辩材料；提交前由本人确认实际完成内容。
代表性提交	（待填写）
工作量占比（%）	25
本人签名	（待填写）

成员 4

项目	内容
姓名	路茗淳
学号	（待填写）
角色	AI 语义、交互与交付
负责模块	显式语义筛选/分类/聚合算子、批处理与缓存协议；浏览器工作台、命令行 REPL、HTTP 安全校验、AI 接入展示、打包、测试计划与报告组织；创新点为受预算约束的 AI 语义执行算子
负责源文件	auroradb/semantic.py、auroradb/ai.py、auroradb/server.py、auroradb/__main__.py、auroradb/web/、tools/semantic_experiment.py、tools/verify.py、tools/build_report.py、tools/build_se_docs.py、docs/
对应测试	tests/test_semantic.py、tests/test_server.py、tests/test_advanced_server.py、tests/test_ai.py、tests/test_ai_deadlines.py、tests/test_ai_integration.py、tests/test_submission.py、docs/semantic_research.json、docs/test_results.json
实际完成内容	拟负责 AI 语义与交互交付组件的核验；与全组共同参与需求分析、构思讨论、四项创新接口评审、界面验收、测试复现、报告与答辩材料；提交前由本人确认实际完成内容。
代表性提交	（待填写）
工作量占比（%）	25
本人签名	（待填写）

7.3 小组确认

项目	内容
工作量合计	100%
组长	钟国鹏
确认日期	2026-10-24（提交前由四位成员共同确认）
备注	四人均参与构思、讨论、需求分析、代码/结果核验、测试复现、报告撰写与答辩准备；模块负责人表示组织和核验责任，不替代成员本人对“实际完成内容”的确认。四人拟定工作量占比合计 100%。